UNIVERSIDAD AUTÓNOMA "GABRIEL RENÉ MORENO"

FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES



Ampliación de una ciudad interactiva con OpenGL

Nombre: Cristian Fernando Huari Choque

Registro: 216024013

fecha: 10-10-2026

Materia: Programación Gráfica

Sigla: ELC102

Grupo de materia: SA

Tecnologías: Java 17 | Maven | LWJGL 3.3.3 | GLFW | OpenGL 3.3 Core

Santa Cruz – Bolivia

1. Resumen

Se amplió el proyecto de referencia (una ciudad de 7 × 7 celdas con auto, iluminación, minimapa y entregas) a una ciudad de **11 × 11 celdas**, conservando el tamaño de cada celda (10 × 10 unidades).

La ampliación no se limitó a dibujar más manzanas. El tamaño del mapa se declara una sola vez, en la matriz `MAPA`, y de ella se derivan el suelo, los límites de circulación, las colisiones, las cámaras, la escala del minimapa, la posición inicial, los destinos y las farolas. Así no quedan números del mapa antiguo escondidos en el código ni en los shaders.

Resultado en cifras:

- **25 manzanas:** 18 con edificios y 7 parques (4 de ellos nuevos).
- **34 farolas**, 25 de ellas en los sectores añadidos.
- **5 destinos de entrega** repartidos por la ciudad, todos accesibles por calle.
- **4 vehículos autónomos** y una interfaz superpuesta (mejoras opcionales).
- **10 pruebas automáticas** que verifican los requisitos principales.

2. Diseño de la ciudad

`MAPA` es una matriz de 11 × 11: **0 = calle, 1 = edificio, 2 = parque**. Las calles ocupan las filas y columnas pares, y las manzanas las impares. Las filas y columnas 0 a 6 son el mapa original, sin cambios; las 7 a 10 son la ampliación (celdas más claras en la figura).

	0	1	2	3	4	5	6	7	8	9	10
0											
1		E		E		P		E		E	
2											
3		P		E		E		P		E	
4											
5		E		P		E		E		P	
6											
7		E		E		P		E		E	
8											
9		E		P		E		E		E	
10											

Figura 1. Mapa 11 × 11. Gris = calle, azul (E) = edificio, verde (P) = parque. Los números son índices de fila y columna.

La ciudad se divide en cuatro sectores, cada uno con un tono de acera distinto:

Sector	Filas / columnas	Contenido	Acera
Casco Antiguo	Filas 0–6, col. 0–6	Mapa original de 7 × 7	Gris azulado
Distrito Este	Filas 0–6, col. 7–10	Manzanas y un parque nuevos	Arena
Distrito Sur	Filas 7–10, col. 0–6	Manzanas y dos parques nuevos	Verde azulado
Barrio Nuevo	Filas 7–10, col. 7–10	Manzanas nuevas	Lila

Red vial. Todas las calles forman una única red conectada. Una prueba automática la recorre y comprueba que el punto de inicio y los cinco destinos son alcanzables. **Variaciones.** La altura de los edificios va de 5 a 13 unidades y su color depende de la posición en el mapa.

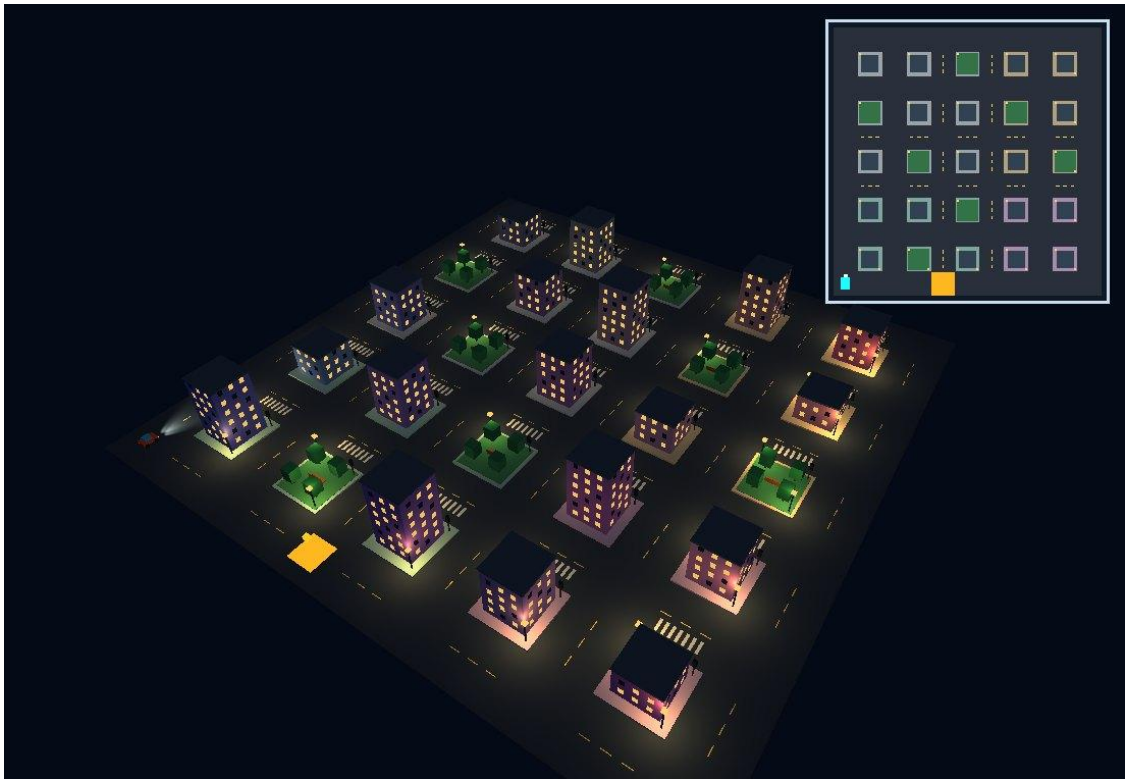


Figura 2. Vista aérea nocturna de la ciudad ampliada, con el minimapa en la esquina superior derecha.

3. Organización del código

El código se organiza por responsabilidad. Cada clase hereda de la anterior y añade su parte, de modo que cada etapa puede ejecutarse por separado.

Clase	Responsabilidad
`clase1`	Ciudad, mapa, cámaras, renderizador de cajas, shaders base y ciclo de vida (`iniciar`, `loop`, `limpiar`)
`clase2`	Conducción con tiempo entre cuadros, colisiones con la ciudad, modelo del vehículo y título de la ventana
`clase3`	Iluminación: día y noche, farolas, faros del auto y cálculo de luz en el shader
`clase4`	Decoración, semáforos, entregas, reinicio y minimapa. Versión completa de los requisitos obligatorios
`clase5`	Mejoras opcionales: tráfico autónomo e interfaz superpuesta
`VehiculoAutonomo`	Lógica de los vehículos autónomos (rutas, giros, velocidad). No depende de OpenGL
`FuentePixeles`	Fuente de texto de 5 × 7 píxeles para la interfaz. No depende de OpenGL

Para llegar a pantalla, cada objeto se dibuja con `caja(...)`, que envía posición, tamaño y color a la GPU como variables *uniform* y ordena `glDrawArrays`. Todos los objetos (edificios, árboles, autos, farolas) se construyen con ese mismo cubo.

4. Funciones implementadas

4.1 Ampliación y coherencia del escenario

- `LIMITE` se calcula como `FILAS × CELDA / 2` (55), en lugar de un valor fijo. De él dependen el suelo, los bordes y la función `centro()`.
- Las colisiones recorren `MAPA` de forma general, por lo que se adaptan al nuevo tamaño sin cambios adicionales.
- La cámara aérea y su distancia máxima de visión son proporcionales al tamaño de la ciudad.
- El shader del minimapa recibe su escala desde Java mediante el uniform `uEscalaMapa`, en lugar de un número fijo.
- La posición inicial y los destinos se definen por celda del mapa, lo que garantiza que siempre caen sobre una calle.

4.2 Iluminación

- Luz ambiental y direccional, con cielo y ambiente distintos de día y de noche (tecla **N**).
- **34 farolas** generadas desde `MAPA`. Su luz se calcula con las normales de las superficies (difusa), decrece con la distancia y se anula a 30 unidades.
- **Dos faros** delanteros en cono, que siguen la posición y orientación del auto, con alcance limitado (tecla **F**).
- La **emisión** visual (bombillas, ventanas, pilotos) está separada de la **iluminación** que recibe el entorno, que se calcula en el shader.

4.3 Decoración y semáforos

- Árboles y bancos en los parques, ventanas en los edificios y pasos peatonales junto a las manzanas.
- De noche, unas ventanas aparecen encendidas y otras apagadas, con un patrón reproducible.
- Semáforos con ciclo de 12 s (rojo 5, verde 5, amarillo 2), controlados por un reloj propio y desfasados entre manzanas vecinas. Son solo visuales.

4.4 Entregas y reinicio

- Cinco destinos; se completa cada uno al frenar (menos de 1 u/s) a menos de 3 unidades de la marca dorada.
- Al terminar se muestra el tiempo total. La tecla **R** restablece posición, orientación, velocidad, entregas y tráfico, y permite repetir toda la actividad.

4.5 Minimapa

- Vista superior simultánea con toda la ciudad, el auto (posición y orientación), el destino activo y el tráfico, con el norte hacia arriba.
- Se muestra u oculta con **M**. Conserva su esquina y su forma al redimensionar la ventana. Usa `scissor` y `viewport`, por lo que no borra ni altera la vista principal.

4.6 Mejoras opcionales

- **Tráfico autónomo:** 4 vehículos con rutas cerradas por el carril derecho, giros suaves y velocidad propia. Frenan ante un vehículo delante y no atraviesan edificios ni salen del mapa (comprobado con 10 minutos simulados). El jugador no puede atravesarlos.

- **Interfaz:** panel con velocidad, estado de faros y ambiente, entrega, tiempo y nombre del sector; ayuda de controles (**H**), pausa (**P**) y mensaje final. El texto se dibuja con una fuente de píxeles propia.



Figura 3. Cámara de seguimiento con la interfaz: ayuda de controles, panel de estado y minimapa.

5. Controles

Tecla	Acción
W / S o ↑ / ↓	Acelerar / frenar y marcha atrás
A / D o ← / →	Girar
Espacio	Freno
C	Cambiar de cámara (seguimiento / aérea)
N	Alternar día y noche
F	Encender o apagar los faros
M	Mostrar u ocultar el minimapa
R	Reiniciar el recorrido
H	Mostrar u ocultar la ayuda
P	Pausa
ESC	Salir

El título de la ventana muestra, en este orden: velocidad, ambiente, estado de faros, progreso de entregas, tiempo, sector y, al final, la lista de controles.

6. Instrucciones de ejecución

Se requiere JDK 17 o superior, Maven y una sesión gráfica con OpenGL 3.3. Desde la carpeta que contiene `pom.xml`:

Versión final (con mejoras opcionales):

```
mvn compile exec:exec
```

Versión con los requisitos obligatorios (sin mejoras opcionales):

```
mvn compile exec:exec -DmainClass=com.graphics.clase4
```

Pruebas automáticas (no abren ventana):

```
mvn test
```

En macOS el perfil de Maven añade `-XstartOnFirstThread`; si se ejecuta desde un IDE, debe añadirse a las opciones de la JVM. La primera vez, Maven descarga las dependencias.

7. Pruebas automáticas

`ConduccionTest` contiene 10 pruebas que no requieren ventana. Verifican, entre otras cosas:

- dimensiones mínimas, cantidad de edificios y parques, y que el límite se deriva del mapa;
- que la red vial es una sola y que el inicio y los cinco destinos están sobre ella;
- la cantidad de farolas y que no superan la capacidad del shader;
- las colisiones en la zona nueva y en los nuevos bordes;
- el ciclo completo de entregas, el reinicio y la repetición de la actividad;
- que ningún vehículo autónomo sale de la calzada ni se detiene durante 10 minutos simulados.

8. Limitaciones

- No hay sombras: las farolas iluminan las fachadas aunque haya otro edificio por medio.
- Los semáforos son solo visuales y no detienen a ningún vehículo.
- Los vehículos autónomos solo guardan distancia con los que van en su mismo sentido; en los cruces no se ceden el paso.
- Las colisiones usan formas simples (círculos y rectángulos), sin física de choque.
- La fuente de la interfaz solo tiene mayúsculas, dígitos y algunos signos, sin tildes.
- Las ventanas redondean sus figuras a cajas: el renderizador solo dibuja cubos.

9. Recursos externos

- **Proyecto base** *CiudadConduccion* (mapa de 7 × 7), entregado en clase: estructura de clases, renderizador de cajas y shaders de partida.
- **Bibliotecas:** LWJGL 3.3.3 (GLFW y OpenGL) y JUnit 3.8.1.
- **Diseño propio:** fuente de píxeles, rutas del tráfico, distribución de la ampliación y pruebas automáticas.
- **Asistente de IA** (Claude, de Anthropic), usado como apoyo en el desarrollo y en esta documentación